National Textile University, Faisalabad

Name:	Qura tul ain
Class:	CS 5thB
Registration No:	23-NTU-CS-1085
Course Name:	Embedded IOT Systems
Submitted To:	Nasir Mehmood

QUESTION 01

PART A:

SHORT QUESTIONS:

1. **What is the purpose of WebServer server(80); and what does port 80 represent?**

   WebServer server(80) is used when we are creating web server on device. 80 is the standard port for normal web traffic. Port 80 helps to access the webpage easily. When user writes IP address of device in browser, the browser automatically connects using this port. This removes the need to specify anything extra.

2. **Explain the role of server.on("/", handleRoot); in this program.**

   In program, we write **server.on("/", handleRoot);** because it tells web server what should it do when main page is opened. Without this user see an error message.

3. **Why is server.handleClient(); placed inside the loop() function? What will happen if it is removed?**

   server.handleClient(); is placed inside the loop() function to allow web server to check and respond to user request. If it is removed then server will not respond to user request and will not perform its function properly.

4. **In handleRoot(), explain the statement: server.send(200, "text/html", html);**

   In this statement 200 is the HTTP code that shows that request is successful. Text/html indicates browser that HTML or a web page has been sent. html is the real content that appears on webpage mostly a string.

5. **What is the difference between displaying last measured sensor values and taking a fresh DHT reading inside handleRoot()?**

   Showing the last sensor values means the page shows numbers that were checked before. It shows up quickly, but the numbers might be a little old.Taking a fresh sensor reading means the sensor checks again when the page opens. The numbers are new and correct, but it takes a tiny bit longer to show them.

Part B:

Long Question:

Describe the complete working of the ESP32 webserver-based temperature and humidity monitoring system.

ESP32 Wi-Fi Connection Process and IP Address Assignment

The ESP32 first connects to a Wi-Fi network using the SSID and password provided in the code. Once connected successfully, the router assigns an IP address to the ESP32. This IP address is used to access the web server by entering it into a web browser.

Web Server Initialization and Request Handling

Once connected to Wi-Fi, the ESP32 is set to work as an HTTP web server using port number 80. The web server is waiting for an incoming request from a browser client. Once the user opens the IP address on his browser, it will receive an appropriate response as per the created request handlers in the function. The function to process an incoming request is always running in this loop to provide an instant response to every situation.

Button-Based Sensor Reading and OLED Update Mechanism

A push button is used to trigger sensor readings. When the button is pressed, the ESP32 reads temperature and humidity values from the DHT sensor. These values are then updated and displayed on the OLED screen, allowing local monitoring without using the web page.

Dynamic HTML Webpage Generation

ESP32 will create a simple HTML page in the code. This page contains the latest readings of temperature as well as humidity. When the browser calls for the page, ESP32 will transmit this HTML, which is displayed like any other page.

Purpose of Meta Refresh in the Webpage

The meta refresh tag is utilized in such a way that the webpage is reloaded automatically after a certain time period. The sensor values will be refreshed automatically without any manual reloading of the webpage by the user.

Common Issues in ESP32 Webserver Projects and Their Solutions

Wi-Fi connectivity failure, the webpage not loading, or an incorrect reading from the sensors are some common issues. These issues can be resolved by making sure Wi-Fi details are correct, the server is being handled properly in the loop, the proper library for the sensors is used, and blocking delays are not used for lengthy periods in the code.

QUESTION 02:

SHORT QUESTIONS:

1. **What is the role of Blynk Template ID in an ESP32 IoT project? Why must it match the cloud template?**

   Blynk Template ID in ESP32 IoT project connect ESP project to specific dashboard template in Blynk cloud. It must match the cloud template so device determines which widgets and settings to use.

2. **Differentiate between Blynk Template ID and Blynk Auth Token.**

   Blynk template ID figure out which project layout the device refers to. Blynk Auth Token is a unique key which enables a certain device to connect safely to project.

3. **Why does using DHT22 code with a DHT11 sensor produce incorrect readings? Mention one key difference between the two sensors.**

   Different timing and data formats are used by DHT11 and DHT22. So when we use DHT22 code with DHT11 sensor cause errors. Main difference between these two is that DHT22 is more accurate and support wide range than DHT11.

4. **What are Virtual Pins in Blynk? Why are they preferred over physical GPIO pins for cloud communication?**

   Software based pins that are used to send and receive data between ESP32 and Blynk app are called virtual pins. These are flexible and does not depend on hardware pins. These are flexible and function properly for cloud communication.

5. **What is the purpose of using BlynkTimer instead of delay() in ESP32 IoT applications?**

When we use delay() it stops the entire program that leds to connection issues while BlynkTimer allow tasks run at set intervals without blocking program and keep the ESP32 responsive and connected to cloud.

Part-B:

Long Question

Explain the complete workflow of interfacing ESP32 with Blynk Cloud to display temperature and humidity values.

Creation of Blynk Template and Datastreams:

A Blynk Template is first created in the Blynk Cloud. Data streams of temperature data, humidity data, among others, are integrated into the template based on the kind of data to be processed. Data streams in this context are the channels to which the ESP32 will make use in the delivery of information back to the Blynk Cloud.

Role of Template ID, Template Name, and Auth Token:

The Template ID specifies the ESP32 device connected to the template in the cloud, thereby ensuring the device is connected to the appropriate interface. The Template Name is used in the identification of the project, although it is primarily for the designer. The Auth Token is a kind of identifier that gives a distinct ESP32 device the ability to connect to the Blynk Cloud securely. The Template ID and Auth Token should be accurately placed within the program.

Sensor Configuration Issues (DHT11 vs DHT22)

The temperature and humidity parameters are read using the ESP32 through the DHT sensor. Both the DHT11 and the DHT22 sensors take different timings to read the values, and the data is presented in different forms. If the code designed to read the values of the given sensor is wrongly applied, it may result in inaccurate results. The DHT22 has a more precise measurement and measures the temperature and humidity levels compared to that of the DHT11.

Sending Data Using Blynk.virtualWrite()

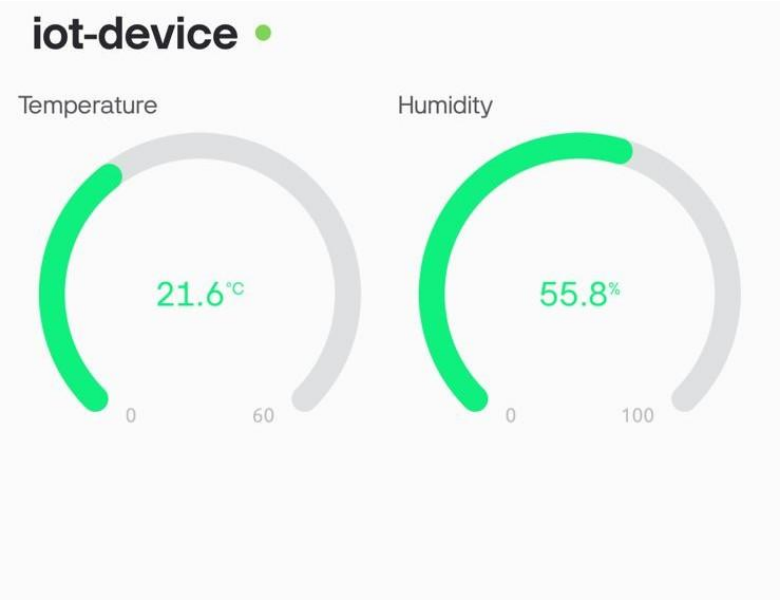
These are then sent to the Blynk Cloud via Blynk.virtualWrite(Vx, value);, where Vx is the respective virtual pin of the value being transmitted. This updates the widgets on the dashboard with the current values of temperature and humidity.

Common Problems Faced During Configuration and Their Solutions

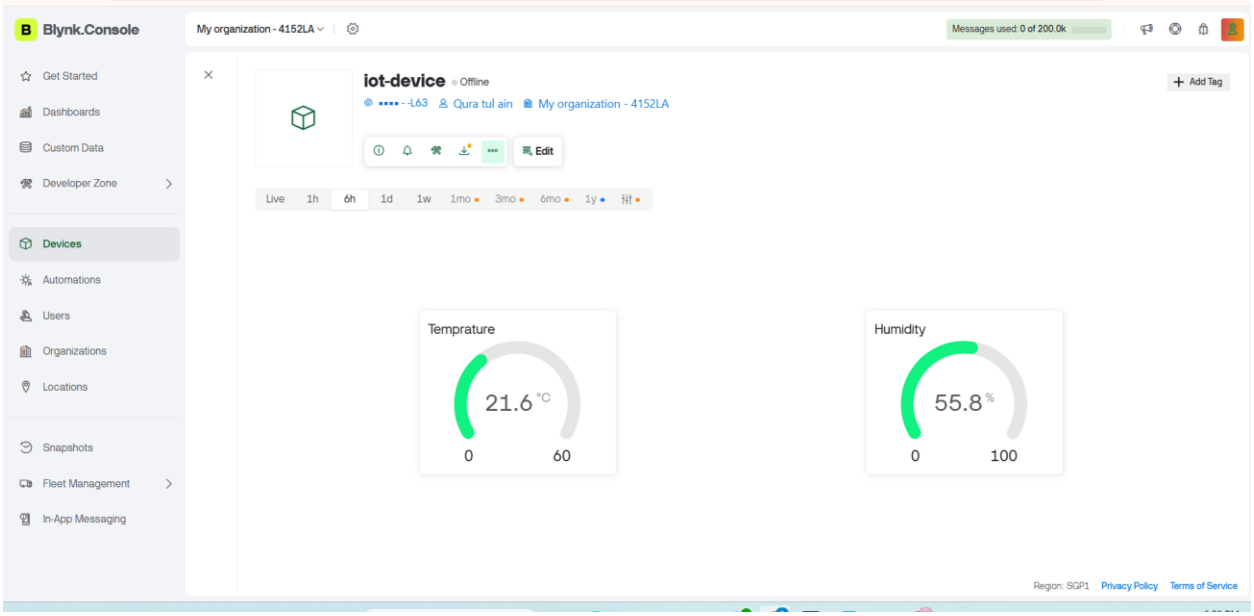
Examples of common problems that can arise with the app are Wi-Fi connectivity problems, the Template ID or the Auth Token being incorrect, incorrect readings from the sensors, or no updates being seen in the widgets from the dashboard. These can be fixed using the following:

- Verifying Wi-Fi Network Credentials
- Verifying use of correct Template ID and Auth Token
- Using correct sensor type and library
- Avoiding long blocking delays in the code to keep the device responsive

MOBILE APP:



WEB DASHBOARD:



Github Repository link:

https://github.com/Qurat123567/Embedded---IOT-Systems.git